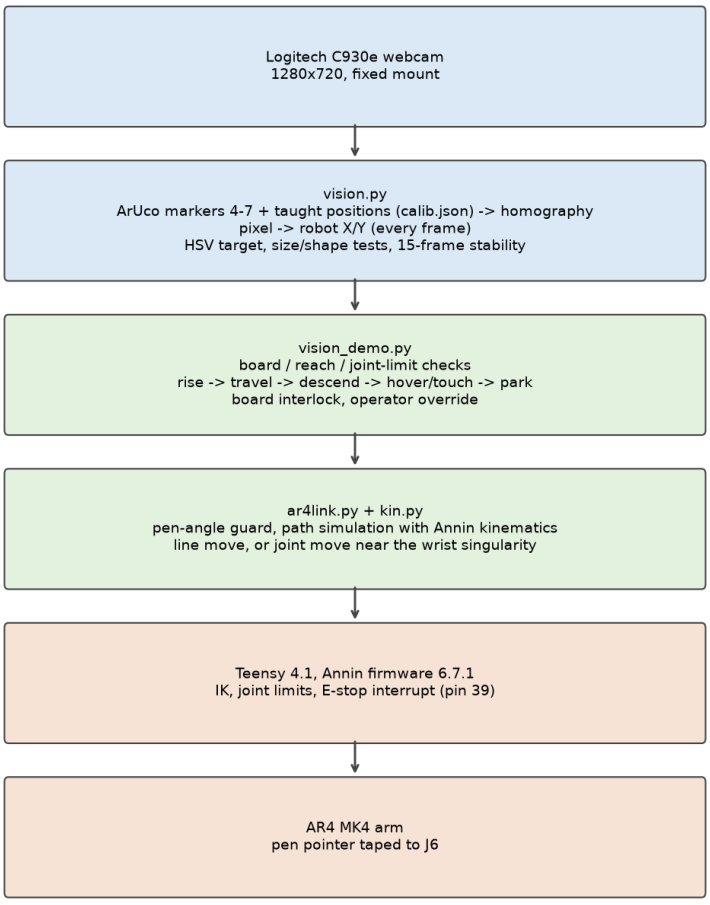
Vision-Guided AR4 Chess – Checkpoint 1

Team 2: Brandon Newton, Roshan Padhan, Andrew Overton
Repository: https://github.com/BengalPirate/AR4_MK4_Vision-Guided_AR4-_Movement
Video: https://www.youtube.com/watch?v=LK-3bo5Rrmg (camera-guided moves to different targets)
Robot: Annin AR4 MK4, Teensy 4.1, Annin firmware 6.7.1 **Camera:** Logitech C930e (1280×720) **Date:** November 9, 2026

Summary. A fixed webcam sees the board with four ArUco markers. The program finds a flat pink target in the live image, maps its pixel position to robot coordinates through a homography taught by touching the markers with the pen, checks the pose is safe and reachable, and moves the physical AR4 so the pen pointer hovers 15 mm above the target (and optionally touches it). Moving the target moves the commanded position. Uncertain detections and unreachable targets are refused. Across 8 target locations the median positioning error was 3.9 mm.

1. System

Data flows top to bottom. All motion runs at 10 % speed. The firmware is unmodified; all checks are in our Python code.



2. Calibration and camera-to-robot mapping

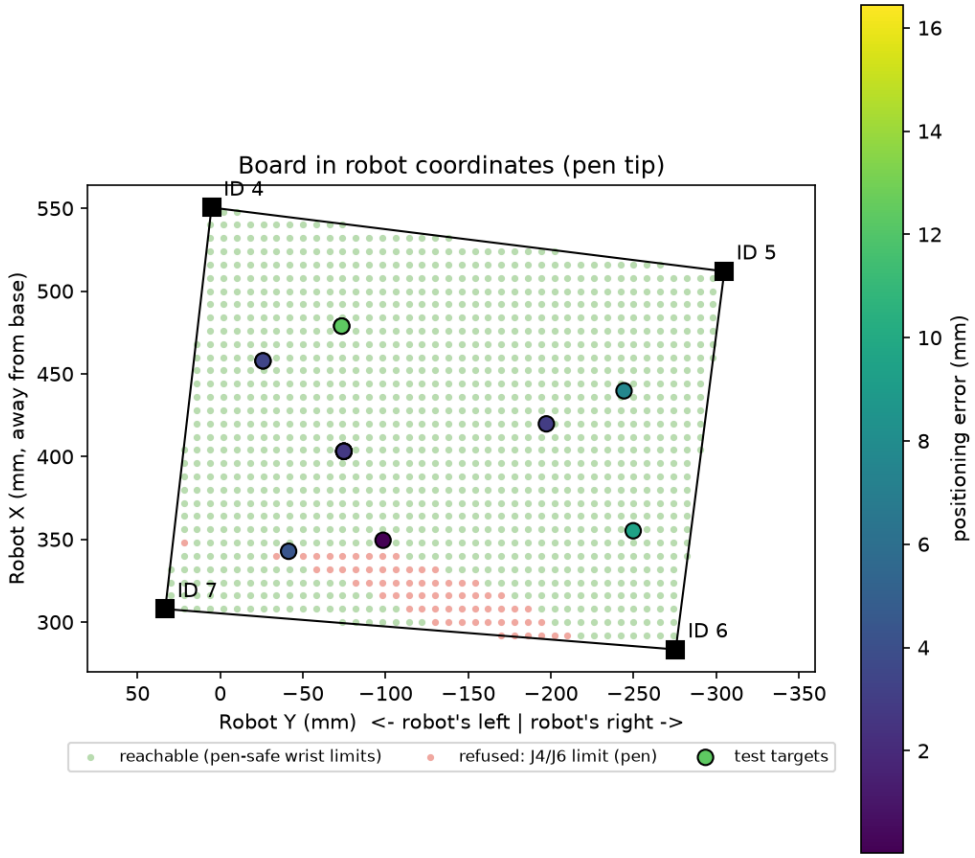
Arm calibration. Annin AR4 HMI 6.7 Auto Calibrate (limit switches), then the HMI is closed.

Board calibration (teach.py). The pen is held at a fixed angle (Rz 0°, Ry 170°, Rx 0°) and jogged until its tip just touches the centre of each marker. The robot X/Y/Z is saved for markers 4–7, plus a PARK pose above the board. Because the pen angle never changes, the flange-to-tip offset is a constant already contained in the taught points: the pen length is never measured, and no tape-measured marker layout is needed.

Mapping (vision.py). In every frame the four marker centres are detected (OpenCV ArUco, DICT_4X4_50) and a homography H maps their pixels onto the taught robot X/Y. The board is a plane, so H maps any board pixel to robot X/Y. Recomputing H each frame tolerates small camera bumps. Board height is a least-squares plane z = a·x + b·y + c through the four touch heights; hover and travel heights are measured from it. The target centre is the centroid of its outline after projection into robot coordinates (no perspective bias).

Marker	Robot X (mm)	Robot Y (mm)	Touch Z (mm)	Plane residual (mm)
4	550.5	5.2	100.0	+0.66
5	511.9	-304.9	100.2	-0.70
6	283.5	-275.5	106.6	+0.71
7	307.9	33.2	103.9	-0.67

Validation of the mapping. Each taught marker, seen by the camera and mapped through H, returns its taught robot position to within 0.1 mm (H is exact at the four markers). The live error at other points is measured below. A second teaching session was also tried; against the same measured points it gave 15–18 mm errors (one marker about 23 mm off), so the first was kept.

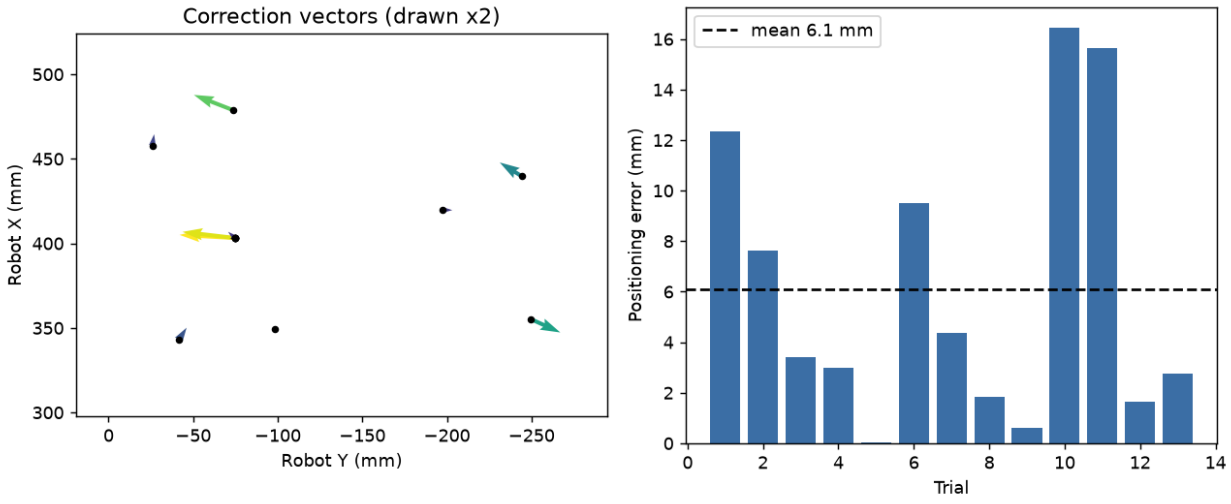


3. Test results

Method. For each trial the target was placed on the board, the program moved the pen above it (G), and the operator nudged the pen in 1 mm steps until the tip was exactly over the target centre, then recorded it (E). The nudge is the positioning error. Every move is logged in results/moves.csv.

Measure	Result
Board plane fit (4 markers)	max residual 0.71 mm
Commanded vs. controller-reported position (17 hovers)	max 0.97 mm
Detection spread when a move starts	max 0.7 mm
Positioning error, 8 target locations	median 3.9 mm, mean 5.3 mm, max 12.3 mm
Repeatability at (403, -75) mm, 6 repeats	commanded spread 0.07 mm; errors 1.8, 0.6, 16.4, 15.7, 1.6, 2.8 mm

#	Target X	Target Y	Correction dX	dY	Error (mm)
1	478.9	-73.5	+4.5	+11.5	12.3
2	439.7	-244.2	+4.1	+6.5	7.6
3	457.8	-25.9	+3.4	-0.5	3.4
4	419.8	-197.2	-0.1	-3.0	3.0
5	349.4	-98.5	-0.0	+0.0	0.0
6	355.2	-249.7	-3.9	-8.6	9.5
7	342.8	-41.4	+3.7	-2.4	4.4
8	403.2	-74.8	-0.2	+1.8	1.8
9	403.2	-74.8	-0.2	+0.6	0.6
10	403.2	-74.8	+0.9	+16.4	16.4
11	403.2	-74.8	+1.8	+15.5	15.7
12	403.2	-74.8	-0.1	+1.6	1.6
13	403.2	-74.8	+1.8	+2.1	2.8



Interpretation. The arm reaches what it is commanded to within 1 mm, and the detection is steady to 0.5 mm, so the error comes from the calibration (how exactly the pen tip was centred on each marker while teaching) and the taped pen. Two repeats at one location needed about 16 mm in +Y although the commanded and reported positions were identical to 0.1 mm; the other four repeats were within 2.8 mm. The cause is not confirmed (pen shifting in its tape, J1 backlash, or an operator nudging error) and is listed under remaining work.

4. Safety checks and failure handling

Layer	Behaviour
Speed	All moves at 10 %; the serial library refuses more than 25 %.
Uncertain detection	Refused unless all 4 markers are visible, exactly one target is on the board, its size (150–6000 mm²) and shape (solidity ≥ 0.85) are plausible, and 15 frames agree within 3 mm.
Workspace	Target at least 15 mm inside the marker rectangle; nothing below the board plane; travel at 100 mm (or the highest reachable of 80/60/40 mm).
Reach and joint limits	Every straight-line move is simulated in 2 mm steps with Annin's kinematics before it is sent; out-of-reach or out-of-limit moves are refused. J4 ±45° and J6 –90…+45° protect the taped pen.
Wrist singularity	Where a straight line would make the firmware swing J4/J6 half a turn, the same pose is reached by a joint move using the wrist solution nearest the current one.
Pen-angle guard	Moves that would also turn the pen by more than 5° are refused (avoids a full wrist sweep caused by Euler-angle wrap near Ry 180°).
Firmware	Unreachable / out-of-limit poses refused by the controller (ER / EL) without motion.
Software stop	SPACE cancels the rest of the sequence before the next move.
E-stop	Hardware interrupt stops the move; the program detects the EB flag. Recovery: release, R (retreat up), P (park).
Interlock	Exclusive board lock: a second arm controller is refused until the first parks.
Operator override	Third repeat of a refused action within 20 s is sent anyway; logged. Firmware limits still apply.

Limit test before camera moves (check_limits.py). Off-board, 700 mm out-of-reach, base-axis and below-board points were rejected in software; an unreachable pose sent to the firmware was refused with ER and no motion; the pen then hovered over markers 4, 5 and 6 with commanded vs. reported differences of 0.37, 0.13 and 0.09 mm, and parked.

Logged refusals and stops during the session. The 'wrist singularity' refusals before 12:30 come from an earlier version that refused near J5 = 0; it was replaced by the joint-move fallback and the pen-safe J4/J6 limits. At 13:07 the operator override was used to reach a target in the joint-limit band.

Time	Event	Result	Reason
11:53:22	limit_test	refused	unreachable 900,0
11:54:24	limit_test	refused	unreachable 900,0
11:55:08	limit_test	refused	marker 7: wrist singularity, J5 5 deg
12:03:02	go	rejected	Unreachable (wrist singularity, J5 11 deg)
12:03:09	go	rejected	Unreachable (wrist singularity, J5 11 deg)
13:05:18	go	rejected	No target (size 18 mm2)
13:05:23	go	rejected	No target (size 28 mm2)
13:07:38	go	rejected	Unreachable (joint limit)
13:07:41	go	rejected	Unreachable (joint limit)
13:07:42	go	override	Unreachable (joint limit)
13:14:35	go	rejected	Board markers not visible: [4]
13:15:06	go	rejected	Ambiguous: 2 targets of this colour
13:15:23	go	rejected	No target (size 77 mm2)
13:15:32	go	rejected	No target (size 77 mm2)
13:15:59	Moving to target	stopped	
13:16:21	Moving to target	estop	Move interrupted, alarm flag 'EB' (E-stop pressed?). Release the E-stop before moving again.

Time	Event	Result	Reason
14:14:38	go	rejected	No target (size 13 mm2)

5. Remaining work (checkpoint 2 and final)

• Replace the colour target with chess-square detection: an 8×8 grid laid over the board through the same homography, and visual piece identification.
• Fit the servo gripper and teach grasp heights per piece; re-teach the board with the gripper's tool point.
• Improve calibration accuracy: teach more points (marker corners), or run the camera-loop refinement (refine.py) from measured trials, then validate.
• Find the cause of the two 16 mm repeat outliers: mark touch dots on paper, check J1 backlash and secure the pen rigidly.
• Chess program (move generation) and game flow for the November 19 demonstration.

6. Team member contributions

Member	Part of the system they explain and operate
Brandon Newton	Calibration and teaching (jogging the pen onto the markers, board-plane fit, PARK); demo operation (live runs, error and repeatability trials).
Roshan Padhan	Camera: mounting and aiming so all four markers are visible, ArUco detection, camera-to-board mapping.
Andrew Overton	Safety testing: limit test, workspace and joint-limit checks, rejection of uncertain or unreachable targets, software stop, E-stop and recovery.

7. External code and AI assistance

• Annin Robotics firmware 6.7.1 and the AR4 HMI 6.7 kinematics module, unmodified (Annin Open Source Non-Commercial License).
• Board and marker approach adapted from another team's Milestone 4 work (github.com/MosesJames46/Embedded-Systems-ARM4-MK4); our implementation is separate.
• ar4link.py and the jog tool extend our writing project (github.com/BengalPirate/AR4_MK4_Team-Writing).
• The code and documentation were written with Claude (Anthropic) in Claude Code, directed by the team. Testing was done on the physical robot: the limit test, live camera-guided trials, and each failure case in section 4, all recorded in results/moves.csv. Issues found in testing (wrist sweep at Ry 180°, wrist singularity near marker 7, pen-safe wrist limits, reach with the opposite pen tilt) were fixed and re-tested on the arm.